TRELLIS.2: licences, weights, GPU, costs, and everything before deployment

TRELLIS.2 is the Microsoft model that turns a machine's 360 video into its 3D model, run as a GPU job in Flex's AWS. This document gathers everything that has to be settled or done before it can make models in production, and what it will cost:

1. **Section 1:** where each item stands.
2. **Section 2:** licences.
3. **Section 3:** access to the model weights.
4. **Section 4:** the GPU to run on in Sydney.
5. **Section 5:** the steps before the first deployment.
6. **Section 6:** costs.
7. **Section 7:** how to measure and reduce them.

The detailed commands for each step are in `docs/deployment-guide.md` (Part 5) and `docs/runbooks/trellis2-worker.md`; this document refers to them rather than repeating them.

What is built. The worker and its image (`gpu/trellis2/`), the GPU environment (`infra/lib/gpu-stack.ts`), the backend's side, the reviewer's screens, the alarms and the image pipeline. **What has not happened yet:** none of it has run on a real GPU or in a real AWS account, and the image has never been compiled. Every GPU timing below is therefore an estimate until the first golden test and the first real builds are measured (section 7).

---

1. Where each item stands

#	Item	State	Who	Blocks
1	**The GPU type.** `g6e.xlarge` (L40S), which the code names, is **not offered in Sydney** (section 4)	Decision needed	Tejesh, with management (cost)	Any GPU deployment
2	**nvdiffrast licence (O-07):** non-commercial use only (section 2)	Open	Legal, management	Models for customers (prod)
3	**DINOv3 access** on Hugging Face (section 3)	Not requested	Someone who can bind the company	Copying the weights
4	**GPU quota** in dev, staging and prod (section 5)	Not requested	Flex's AWS team	Any GPU job
5	**Real AWS account numbers** in `infra/cdk.json`	Placeholders	Thomas, Flex's AWS team	Any deployment
6	**The first image build**, never compiled yet	Not started	Tejesh	The first GPU job
7	**Weights copied** into each environment's bucket	Not started	Tejesh	The first GPU job

#	Item	State	Who	Blocks
8	**The golden reference** recorded from the first image	Not started	Tejesh	Testing new images automatically

Items 1 to 4 wait on decisions, people or AWS, so start them first, side by side.

---

2. Licences

Everything TRELLIS.2 uses

Part	What it is	Licence	Commercial use
TRELLIS.2 code (`microsoft/TRELLIS.2`)	The model's code	MIT	Yes
TRELLIS.2-4B weights	The model	MIT	Yes
TRELLIS-image-large (one decoder)	A weight file TRELLIS.2 reuses	MIT	Yes
DINOv3 (`facebook/dinov3-vitl16-pretrain-lvd1689m`)	The image encoder	DINOv3 License	Yes, once accepted (below)
BiRefNet (`ZhengPeng7/BiRefNet`)	Background removal, used **in place of RMBG-2.0**, which is non-commercial (CC BY-NC 4.0) and is not used	MIT	Yes
CuMesh, its cubvh submodule, FlexGEMM, utils3d	Mesh and sparse-convolution libraries	MIT	Yes
o-voxel's Eigen	Linear algebra, header-only, unmodified	MPL 2.0	Yes
flash-attn	Attention kernels	BSD 3-Clause	Yes
PyTorch; the CUDA runtime in the base image	Framework and GPU runtime	BSD-style; NVIDIA's CUDA licence	Yes
nvdiffrast (`NVlabs/nvdiffrast`)	Texture baking in the GLB export	NVIDIA Source Code License, "1-Way Commercial"	**No**: non-commercial research or evaluation only (section 3.3)

`gpu/trellis2/README.md` ("Licence-safe configuration") records how the configuration avoids RMBG-2.0, and the pins of each part.

DINOv3: accept the licence

Read at the pinned revision, the DINOv3 License says:

- commercial use is allowed;

- it must be accepted by someone who can bind the company, which is what the gated access request on Hugging Face does;
- research publications must acknowledge DINOv3;
- no "Built with DINOv3" credit is required.

The weights are not redistributed: they run in our GPU job, and the licence file is kept beside them.

Cost: none.

nvdiffrast: the one blocker

TRELLIS.2's GLB export uses nvdiffrast to bake textures. Its licence allows use only for non-commercial research or evaluation, other than by NVIDIA. Making models for paying customers is commercial use.

Option	What it takes	Cost
1. Legal decides our use is allowed as it stands	Legal reads the licence	Legal's time
2. A commercial licence from NVIDIA	A request through NVIDIA's research licensing enquiries	Set by NVIDIA, on request; not published
3. Replace nvdiffrast in the texture bake	The 3D team rewrites the worker's export step without nvdiffrast, with its own task and tests	Engineering time; no licence fee

Meanwhile:

- Ask Legal, in the same request, whether internal evaluation on dev, with our own test recordings and not customer data, counts as "evaluation".
- Prod waits for the decision.

Done when the decision is recorded in O-07, and if it is option 3, the replacement is built.

---

3. Access to the weights

The weights (32 files, about 18 GB) come from Hugging Face. They are locked by revision and SHA-256 in `gpu/trellis2/weights.lock.json`. Today 29 files are locked; DINOv3's three are under `pending`, because they cannot be read without access.

The Hugging Face token never goes into AWS. It is used twice, on a machine you control:

- once to lock DINOv3's checksums;
- once per environment to copy the weights into that environment's own S3 bucket.

GPU jobs read the weights only from that bucket, with Hugging Face switched off. Keep the token in the company's password manager, never in the repository, `cdk.json`, Secrets Manager or the image.

Steps (details in `deployment-guide.md`, 5.2, 5.4, 5.5):

1. **Request access.** A company Hugging Face account requests DINOv3 access at https://huggingface.co/facebook/dinov3-vitl16-pretrain-lvd1689m, accepting the licence (section 2). Meta approves by hand.
2. **Make a token.** Create a Read token on that account, or a fine-grained one with "Read access to contents of all public gated repos you can access". Check it, with the command in the guide.
3. **Lock DINOv3's checksums:** `python lock_weights.py` in `gpu/trellis2`, with the token. Run the tests, then commit the lock. The push starts an image build, because the lock goes into the image.
4. **Copy the weights into each environment's bucket**, dev first: `python3 publish_weights.py s3://<models-bucket>/gpu/trellis2/weights/ --work <folder>`, with the token. It checks every file against the lock, uploads them, and writes `READY` last.
    - It needs about 20 GB of disk, and permission to write that bucket and use its KMS key. The policy is in the guide.
    - An EC2 instance in that account in Sydney is quickest.
5. **Afterwards:** delete the work folder, and stop the instance. The token can be revoked until it is next needed (a new lock, or a new environment).

Cost: Hugging Face and DINOv3 are free. The copy machine costs under $1 per environment (section 6).

---

4. The GPU in Sydney

What changed

The code, the guide and the earlier plan chose `g6e.xlarge` (one NVIDIA L40S, 48 GB) in Sydney. **AWS does not offer G6e in Sydney:**

- AWS's on-demand price list for Sydney has no `g6e` size at all;
- AWS lists G6e only in US East, US West (Oregon), Tokyo, Seoul, the UAE, Frankfurt, Spain and Stockholm.

Customer data must stay in Sydney (`docs/residency.md`), so the work cannot move to one of those regions. The GPU type has to change.

The options in Sydney

On-demand list prices, USD an hour:

Instance	GPU (memory)	vCPU, RAM	Local disk	Price	Runs today's image?
`g5.2xlarge`	NVIDIA A10G (24 GB)	8, 32 GiB	450 GB NVMe	**$1.5758**	Yes: the image is built for Ampere (8.6)
`g6.2xlarge`	NVIDIA L4 (24 GB)	8, 32 GiB	450 GB NVMe	**$1.2711**	Yes: Ada (8.9), like the L40S
`p5.4xlarge`	NVIDIA H100 (80 GB)	16, 256 GiB	3.84 TB NVMe	$8.944	No: needs the image rebuilt for Hopper (9.0)

Instance	GPU (memory)	vCPU, RAM	Local disk	Price	Runs today's image?
`g5.xlarge`, `g6.xlarge`	24 GB	4, 16 GiB	250 GB	$1.31, $1.05	Too little RAM: the job needs 28 GiB

Memory. 24 GB is TRELLIS.2's stated minimum; the RunPod kit recommended 48 GB. At the worker's defaults (resolution 1024, a 2048 texture), 24 GB is expected to fit, but with little room. The highest resolution (1536) may run out of memory.

Recommendation:

1. Use **`g5.2xlarge`**. The A10G has twice the memory bandwidth of the L4, which usually makes this kind of model faster, so it should cost the same or less per model despite the higher hourly price.
2. Run the first golden test on both `g5.2xlarge` and `g6.2xlarge` (same image) and keep whichever is cheaper per model without running out of memory.
3. Move to `p5.4xlarge` only if 24 GB proves too small. It costs about six times as much an hour, and needs the image rebuilt for Hopper.

What changes in the code (Tejesh, once the type is chosen):

- `infra/lib/gpu-stack.ts`: the instance type, 8 vCPUs a host, and the 28 GiB job memory (which fits 32 GiB);
- its tests;
- the GPU quota numbers: 8 vCPUs a host, not 4;
- the guide and the diagram.

---

5. Everything before the first deployment, in order

Each step names its owner, and where the guide (`docs/deployment-guide.md`) has the detail.

Step	What	Who	Guide
1	Choose the GPU type (section 4), and change the GPU stack to it	Tejesh, with management	—
2	Ask Legal about nvdiffrast (section 2)	Tejesh, Legal	5.1
3	Request DINOv3 access (section 3)	The company	5.2
4	Request the GPU quota, "Running On-Demand G and VT instances" (`L-DB2E81BA`), in each environment account. Hosts have 8 vCPUs: ask for at least 16 in dev and staging and 32 in prod, to leave room.	Flex's AWS team	5.3
5	Put the real account numbers in `cdk.json`; bootstrap the accounts; deploy the pipelines and the environments, without a GPU yet	Thomas	2.1 to 2.6
6	Lock DINOv3's weights and push	Tejesh	5.4
7	Copy the weights into each environment's bucket	Tejesh	5.5

Step	What	Who	Guide
8	Build the first image through the image pipeline, and fix the build if the first compile fails	Tejesh	5.6
9	Turn on the GPU environment in dev with that image	Tejesh	5.7
10	The first golden test on dev; look at the model; record the reference	Tejesh	5.8
11	A real machine, end to end, on dev, with our own recordings	Tejesh, a reviewer	5.9
12	Staging: weights, image, golden test	Tejesh, Thomas	5.10
13	Prod, once nvdiffrast is settled	Thomas, an approver	5.11

---

6. Costs

Prices used

AWS list prices for Sydney (`ap-southeast-2`), on demand, in US dollars, before tax, at the time of writing. Check the AWS Pricing Calculator before budgeting: prices change.

Item	Price
`g5.2xlarge` (A10G)	$1.5758 an hour
`g6.2xlarge` (L4)	$1.2711 an hour
`p5.4xlarge` (H100)	$8.944 an hour
CodeBuild `general1.2xlarge` (the image build)	$0.25 a minute
CodeBuild `general1.small` (golden test, adopt and trigger steps)	$0.005 a minute
S3 Standard	$0.025 per GB-month
ECR storage	$0.10 per GB-month
`m6i.large` (the weights copy machine)	$0.12 an hour

AWS Batch, EventBridge and the queues cost nothing extra at this volume. A GPU host's 100 GB root disk adds about $0.01 an hour while the host runs.

One-off costs

Item	When	Cost
Image build: two to four hours on the 2XLARGE build host	Each new image	$30 to $60
Golden test: one GPU job, including a cold host	Each new image	Under $1 on `g5.2xlarge` or `g6.2xlarge`; about $4 on `p5.4xlarge`

Item	When	Cost
Weights copy: about an hour on an `m6i.large`, with its disk	Once per environment, and per new lock	Under $1 each
Hugging Face account, DINOv3	Once	Free
nvdiffrast	Once	Depends on the decision (section 2)

Monthly fixed costs (with no models being built)

Item	Cost a month
GPU hosts	$0: none run while idle
Weights: about 18 GB in each of 3 environments	About $1.35
Images in ECR: an estimated 10 to 20 GB each; the repository keeps the last 20	$20 to $40. Keeping 5 instead: $5 to $10
Logs and alarms for the GPU work	A few dollars
Total	**About $25 to $45**, mostly old images

Cost per machine model

A GPU host is billed per second while it runs. Two kinds of time make up a build:

- **Host start (S):** booting, pulling the image, copying the weights to the host's disk, and the idle minutes before Batch stops the host. **Estimate: about 15 minutes**, paid once per host, however many models it then builds.
- **Per model (W):** loading the model, picking frames, the cut-out, TRELLIS.2 at resolution 1024, and the GLB export. **Estimates:**
- about 6 minutes on `g5.2xlarge`;
- about 9 minutes on `g6.2xlarge`;
- about 3 minutes on `p5.4xlarge`.

Cost = hourly price × (S + number of models × W) ÷ 60, for each host that runs.

	`g5.2xlarge`	`g6.2xlarge`	`p5.4xlarge`
One machine on its own (host started for it)	about $0.55	about $0.51	about $2.68
A gym's 40 machines on one host	about $6.70 ($0.17 a model)	about $7.94 ($0.20 a model)	about $20.12 ($0.50 a model)

In prod, `gpuMaxInstances` is 2, so a large batch is split over two hosts. That finishes sooner, at the cost of one more host start (about $0.40 on `g5.2xlarge`).

A machine rebuilt from the phone's depth (the measured route) runs on the same hosts. It uses no GPU and no weights, and takes a few minutes, so it costs about the same as a TRELLIS.2 model or less.

A month at different volumes

This assumes models come in batches, a gym at a time (about 40 machines), plus one rebuild for every five models, each rebuild on a host started for it. It includes the fixed costs above (about $35).

Models a month	`g5.2xlarge`	`g6.2xlarge`	`p5.4xlarge`
200	about $80	about $85	about $210
1,000	about $265	about $285	about $900
5,000	about $1,190	about $1,290	about $4,370

These cover TRELLIS.2 only. The rest of the platform (API, database, storage, networking, web hosting) is outside this document.

---

7. Measuring and reducing costs

Replace the estimates with real figures as soon as there are some:

- Every job's `metadata.json` records its own timings (`timings_s`): frames, cut-out, model loading, generation and export.
- AWS Batch records each job's start and end.
- The route test (`gpu/trellis2/route_test.py`, L-01) reports the time per machine.

Recompute the per-model cost with the formula above.

Follow the spend:

- Every resource is tagged `project: flex-gym-scan` and `environment: <env>`. Turn these on as cost allocation tags in Billing, then filter Cost Explorer by them.
- Set an AWS Budget per account, with an alert, at a level agreed with management.

Reduce it, if needed:

- **Spot instances.** AWS Batch can run the GPU jobs on Spot capacity, usually much cheaper than on demand. An interrupted job is retried (each job allows three attempts). This needs the "All G and VT Spot Instance Requests" quota (`L-3819A6DF`), and a change to the GPU stack.
- **Keep fewer images in ECR**, for example 5 instead of 20.
- **Build machines in batches.** Builds already start as each video arrives, so a gym recorded in one visit shares one host start.
- **Keep `gpuMaxInstances` low:** fewer hosts means fewer host starts. Raise it only if waits grow too long.